UNIVERSIDAD DE CARABOBO
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación
Unidad Académica de Algoritmos y Programación
CAO402: Algoritmos II

Profesor: Álvaro Espinoza

Fecha: 25/06/2025

PARCIAL I: PROGRAMACIÓN ORIENTADA A OBJETOS
Período Lectivo 1-2025

Nombre y Apellido: _____ Cedula de Identidad:_____

PARTE I: Lea cuidadosamente la pregunta y seleccione la opción correcta. Se aplicará factor de corrección: 3 respuestas incorrectas eliminarán una correcta, una respuesta en blanco no afectará a las respuestas correctas (1 punto c/u)

a) Sea una clase A que contiene a una clase B. A su vez, B contiene a una clase C:
   a) Por transitividad, A puede contener elementos de tipo C
   b) Es posible que el profesor quiera confundirnos
   c) A sólo puede contener objetos de tipo B.
   d) La clase C debe estar definida antes que la clase A para que el código compile
   e) A puede contener elementos de tipo C si C es pública

b) Dada una clase W con una asociación "usa" en dirección a una clase A. Y A está compuesta por una clase B.
   a) Yo se programar. Me falta solo el título.
   b) La asociación "usa" permite a W acceder a los atributos privados de B
   c) W y B no están relacionados
   d) W y B están relacionados
   e) La composición entre A y B es más eficiente sustituirla por una herencia

c) Usted está diseñando una clase Pedido, la cual necesita trabajar con objetos de la clase Producto. Un colega le pregunta qué tipo de relación establecer entre ambas clases. ¿Cuál de las siguientes afirmaciones representa mejor una asociación adecuada entre Pedido y Producto?
   a) Debería usar una relación de herencia, ya que un producto es un tipo de pedido.
   b) Una composición es adecuada si Producto debe seguir existiendo aunque se elimine el Pedido.
   c) Si Pedido tiene una lista de productos, eso obliga a usar composición, ya que hay múltiples instancias.
   d) Una asociación de uso es suficiente si Pedido solo necesita acceder a Producto en un método temporalmente
   e) La agregación no es válida porque ambas clases no pueden ser instanciadas por separado

d) Usted ha diseñado una jerarquía de clases donde: La clase Animal define comportamientos generales. La clase Mamifero hereda de Animal. La clase Perro hereda de Mamifero ¿Cuál de las siguientes afirmaciones es correcta respecto al uso de herencia en múltiples niveles?
   a) La clase Perro no puede sobrescribir métodos definidos en Animal porque no hereda directamente de ella
   b) Es preferible que Perro herede directamente de Animal para evitar depender de clases intermedias.
   c) Perro hereda todos los atributos y métodos públicos y protegidos de Animal a través de Mamifero.
   d) Mamifero debe ser abstracta para que Perro pueda heredar de ella correctamente.
   e) No debo rendirme hasta aprobar la materia

e) Una desarrolladora está implementando una clase Calculadora que tiene varios métodos llamados sumar(), cada uno con diferente cantidad o tipo de parámetros. El compilador decide automáticamente cuál método usar según los argumentos que se le pasan al invocar sumar(). ¿Cuál de las siguientes afirmaciones describe correctamente esta situación?
   a) Thanos tenía razón
   b) Se trata de polimorfismo dinámico porque el método sumar puede comportarse distinto en tiempo de ejecución.
   c) Esto es una sobrecarga de métodos, un caso típico de polimorfismo estático
   d) El comportamiento depende de qué clase hereda de Calculadora, por lo tanto es herencia múltiple

e) No hay polimorfismo, ya que todos los métodos tienen el mismo nombre

f) Un estudiante está revisando dos fragmentos distintos de un sistema: En uno, hay una clase Calculadora con varios métodos sumar() que reciben distinta cantidad o tipo de parámetros. En otro, una variable de tipo Figura puede hacer llamadas a dibujar(), y dependiendo de si se trata de un Círculo o un Rectángulo, el comportamiento es distinto en tiempo de ejecución. ¿Cuál de las siguientes afirmaciones describe correctamente ambos casos?
   a) Honestamente creo que me gusta mas el calculo
   b) En ambos casos se aplica polimorfismo estático, ya que los métodos comparten nombre.
   c) El primer caso representa encapsulamiento, y el segundo, herencia
   d) El primer caso es polimorfismo estático (sobrecarga) y el segundo, polimorfismo dinámico (sobreescritura)
   e) Ninguno de los dos casos representa polimorfismo, ya que el comportamiento está completamente definido por el programador

g) Dada una clase A que contiene en agregación a una clase B. Y una clase C que hereda de A y B:
   a) Esta relación no es posible en la Programación Orientada a Objetos
   b) A tiene acceso a los métodos privados de B
   c) A solo puede contener elementos de tipo B
   d) C puede contener objetos de tipo C
   e) Las materias se pelean hasta el final

h) Dada una clase X con cinco (5) constructores polimórficos no heredados:
   a) Esto es una agresión
   b) Esto es posible gracias al polimorfismo dinámico
   c) Esto es posible gracias al polimorfismo estático
   d) Esto es un ejemplo de sobreescritura de métodos
   e) La mala definición de estos constructores es detectada en tiempo de ejecución

i) Sea A una clase abstracta con diez clases hijas.
   a) Solo una de las diez clases hijas puede implementar los métodos abstractos de A
   b) Las clases hijas están obligadas a heredar solo de A, ya que no se permite herencia múltiple con clases abstractas
   c) Si una clase hija no implementa todos los métodos abstractos de A, el programa no compila
   d) No tiene sentido instanciar la clase A al ser abstracta
   e) El profesor disminuirá la población de la materia a la mitad

j) Usted ha diseñado una clase A con todos sus atributos privados. Y se encuentra programando las clases B, C y D que heredan todas de A. Un colega le sugiere cambiar sus atributos a protegidos. ¿Cuál es la razón de esto?
   a) El polimorfismo no permitirá sobreescribir los métodos getters/setters de A en las clases hijas si sus atributos son privados
   b) Al profesor no le gusta que hagan eso
   c) Esto facilitará el trabajo con los atributos de A en las clases hijas ya que al ser protegidos se puede acceder a ellos directamente
   d) Cambiar los atributos a protegidos permitirá que las instancias de B, C y D accedan entre sí a sus atributos internos.
   e) Los atributos protegidos hacen que los objetos de las clases hijas se comporten como si fueran del tipo padre.

PARTE II: Modelado de Clases: Realice el diagrama de clases correspondiente a la siguiente situación **(5 pts)**:

Convención Otaku del Sol Naciente

La organización de la Convención Otaku del Sol Naciente desea registrar y gestionar la participación de sus asistentes. De cada fan se almacena su nombre, número de identificación, fecha de nacimiento, edad y fecha de inscripción al evento.

Cada fan puede participar en distintos roles durante la convención: cosplayers, expositores, coleccionistas, autoadmiradores, multifans o monofans. Es importante destacar que un mismo asistente puede desempeñar múltiples roles, y que cada tipo de participante tiene información asociada diferente dependiendo de su categoría.

Por ejemplo, los multifans son seguidores de dos (2) o más franquicias de anime o manga (como One Piece, Naruto o Evangelion), mientras que los monofans son fieles a una sola. En cambio, los autoadmiradores son fanáticos solo de sus propios personajes originales o diseños de cosplay, por lo que no siguen franquicias externas. Por otra parte, se organizan paneles, talleres y concursos dirigidos por los expositores, que pueden ser uno o varios por actividad. Cada evento tiene una fecha, hora y sala asignada. Los asistentes pueden elegir libremente a cuáles actividades desean acudir.

Se desea mantener un registro claro de qué actividades ha asistido cada fan, así como de qué franquicias siguen o admiran. Además, se requiere llevar un conteo general del número total de fans registrados en la convención.

PARTE III: Implementación (5 pts):

Sistema de Organización de Torneos de Ajedrez

Se desea modelar un sistema básico para organizar torneos de ajedrez en una liga regional. Para ello, deberás diseñar un conjunto de clases que representen los **jugadores**, los **partidos** y el **torneo** en su conjunto.

Requisitos del sistema:

1. Un **torneo** está compuesto por múltiples **partidos**, y debe permitir:
    - Registrar partidos individuales.
    - Calcular el total de partidas jugadas.
    - Obtener la lista completa de jugadores que han participado, sin repeticiones.

2. Cada **partido** involucra exactamente **dos jugadores** y almacena:
    - El identificador del partido.
    - Una referencia a los dos jugadores involucrados.
    - El nombre del ganador (puede ser None si aún no ha terminado).

3. Un **jugador** tiene:
    - Un nombre completo
    - Una categoría (por ejemplo: "Aficionado", "Avanzado", "Maestro").

Indicaciones adicionales:

- Las clases deben estar relacionadas entre sí de forma estructural y obligatoria. Por ejemplo, un partido no puede existir sin jugadores, y un torneo se define por su colección de partidos.
- Es obligatorio declarar los métodos *getters* y *setters* para todos los atributos relevantes. No es necesario implementar su lógica interna, pero deben estar presentes en el código.
- El diseño debe facilitar operaciones básicas del torneo sin necesidad de estructuras externas.

……

Sobró espacio, te regalo un código para que te recuerdes de estudiar arquitectura :)

```
.data
mensaje: .asciiz "Te recuerdo que debes estudiar tambien arquitectura :)\n"
.text
.globl main
main:
    li $v0, 4            # Código de syscall para imprimir cadena
    la $a0, mensaje      # Cargar dirección del mensaje en $a0
    syscall              # Llamada al sistema
    li $v0, 10           # Código de syscall para salir del programa
    syscall              # Salir
```

"Si la depuración es el proceso de eliminar errores, entonces la programación debe ser el proceso de introducirlos" - Edsger Dijkstra